

## 7. Technical Validation and Performance

### 7.1 - Local Datamart and Dashboard Validation

This section summarizes local validation of the January 2025 synthetic dataset and the DuckDB and Streamlit prototype. The results apply only to this setting.

#### Test environment

- dataset: January 2025 monthly billing;
- charge lines: 164,145;
- local analytical engine: DuckDB 1.5.5;
- dashboard framework: Streamlit 1.63.0;
- serving objects: 14 materialized SQL datamarts, including 11 queried by the dashboard.

#### Reconciliation results

The executive datamart and the existing monthly-billing datamart both report a billed cost of EUR 912,000.00. The executive charge-line count and the monthly quality row count both equal 164,145. The quality table reports zero null values for BilledCost, BillingCurrency and ServiceName, and one ingestion batch for the selected month.

#### Application validation

Streamlit AppTest executed the initial FinOps page, four Cost Analysis views and four Data & Architecture views. All eight navigable subviews completed with zero application exceptions and zero Streamlit warning elements. Python compilation completed successfully. The existing automated suites also completed successfully: one synthetic-generator test and nine billing-notebook regression tests.

### Cost-comparison observations

| Indicator                         | January 2025 result |
|-----------------------------------|---------------------|
| List cost                         | EUR 1,290,989.95    |
| Contracted cost                   | EUR 1,044,652.85    |
| Effective cost                    | EUR 1,101,755.75    |
| List minus contracted             | EUR 246,337.10      |
| Contracted minus effective        | EUR -57,102.90      |
| List minus effective              | EUR 189,234.20      |
| List-to-effective difference rate | 14.66%              |

The negative contracted-to-effective difference prevents this component from being accepted as demonstrated commitment savings. The result is retained as an analytical observation requiring semantic validation, not corrected or hidden to produce a favorable KPI.

### Validity boundary

These results demonstrate local technical execution and reconciliation for one synthetic month. They do not demonstrate Databricks SQL compatibility, production performance, enterprise certification, Row-Level Security, actual organizational savings or forecasting capability.

## 7.2 - Existing Power BI Diagnosis and Measurement Protocol

The existing Power BI report provides the professional comparison point for reporting needs. The student's account identifies a multi-step change process, but it contains no measured report latency, refresh duration or capacity data. A slow interaction could arise from the semantic model, its source, a visual, network transfer, shared capacity or cache state. The present evidence does not identify which component dominates.

A diagnosis would fix a representative report page, filter setting, user task and expected answer before timing. Initial loading and repeated interactions should be recorded separately because cache state can change the result. Within authorized access, Power BI Performance Analyzer can separate query, visual display and other elapsed time; model and source inspection would then test a specific cause rather than assume one. Each result needs the report and model version, data volume, connection mode, capacity, hardware, date and number of repetitions. Median and dispersion are more informative than a single selected run.

*Source: Microsoft, Use Performance Analyzer to examine report performance.*

No such measurements are currently indexed. The comparison in section 7.3 therefore treats the existing report as an operational reference and does not claim that the local Streamlit prototype improves its response time. Refresh delay and interactive latency would also need separate measurements because they affect different user tasks.

### 7.3 - Architecture Comparison and Performance Levers

The local POC publishes fourteen materialized datamarts, eleven of which its Streamlit application queries. Section 7.1 establishes their functional execution and reconciliation for January 2025. It does not measure the latency benefit of materialization. An equivalent direct query and datamart query would be needed in the same environment, with identical answers and recorded cache state, to attribute a difference to this design choice.

Cross-system comparisons require further controls. The company's Power BI report and the synthetic local POC differ in data, engine, model, network and capacity. A raw timing difference between them would describe those particular settings rather than establish architectural superiority. Where access permits, query execution, application processing, rendering and end-to-end response should be recorded separately. Storage and refresh cost also belong in the comparison because a materialized answer may be faster to read but more expensive to maintain.

The repository defines Serverless and Classic Databricks scenarios with common processing logic. The earlier Serverless `STANDARD` setting and the Belgium Classic templates differ in compute and Job policy; the reported historical backfill does not resolve those differences experimentally. A controlled comparison would process the same source months and code revision into isolated outputs, verify result equality, and repeat runs with region, runtime, Photon, worker settings, concurrency and cache recorded. Startup time, total duration, DBUs and full provider cost would then be compared. No quantified Serverless or Classic advantage is currently supported by the project evidence.

### 7.4 - Databricks/GCP Validation

#### 7.4.1 Validation boundary

Cloud validation addresses a different claim from the local DuckDB POC: whether the shared Parquet source can be processed by Databricks, isolated into DEV and PROD catalogs, reconciled, and exposed as governed analytical products. The tests should identify the workspace, region, compute type, Photon setting, executed Git commit, source period, Job and Run IDs, and final table versions. A repository file or deployment guide establishes the intended procedure; an execution record establishes what ran. The available plan locates the work in the Belgium workspace and identifies a Classic scenario, but its summary alone does not provide these run-level details.

The acceptance criteria follow the data path. The RAW file must be readable; Bronze must preserve its rows and source lineage; contract failures must prevent invalid Silver publication; the authoritative billing month must replace rather than add to provisional Daily rows; Gold and datamarts must reconcile to Silver; and OPS must retain distinct DEV and PROD histories. The application must use permitted read access without write privileges. These criteria test both functional correctness and governance rather than interpreting a green Job icon as complete validation.

#### 7.4.2 Evidence currently available

The working Belgium completion note states that the four catalogs exist, that a monthly backfill for January 2025-June 2026 and DEV/PROD controls succeeded, and that the first 1 July 2026 Daily file was loaded in DEV with `promote_to_prod=false`. Section 3.2.3 independently reports local validation of eighteen `no_change` billing files through June 2026 and Daily files through August 2026. These source counts can be compared with the cloud tables once the relevant Run IDs and query outputs are captured. The cloud claims remain reported execution awaiting primary artefacts, rather than independently reproduced thesis measurements.

| Capability                   | Current record                                          | Evidence needed for a validated thesis claim                                                                        |
|------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Historical DEV/PROD backfill | Reported successful in the Belgium plan                 | Executed commit; Job/Run IDs; dates; task outputs; rows and <code>BilledCost</code> by month; reconciliation status |
| Daily DEV load               | 1 July 2026 reported in DEV only                        | Run ID; source URI; rows written; DEV checks and month status                                                       |
| Daily promotion to PROD      | Job template and procedure exist                        | Complete DAG run; matching DEV/PROD outputs; OPS records; idempotent replay                                         |
| Monthly close DEV → PROD     | Module and notebook exist; dedicated Classic Job absent | Billing-arrival guard; both runs; <code>BEFORE/SOURCE/AFTER</code> ; equality and failure recovery                  |
| Databricks App               | Source and deployment guide exist                       | Deployment record; page checks; Warehouse and Unity Catalog permissions                                             |

### 7.4.3 Daily-to-close experiment

A complete Daily experiment should begin with a RAW file in an open month. The discovery step must choose the oldest file absent from PROD, then DEV load and validation must succeed before the promotion gate admits PROD. The same URI must reach both environments. A deliberate replay of that file should write no duplicate Bronze or Silver rows while keeping the checks green. A no-candidate run should end without loading data. Because the Job processes one candidate per run, a backlog requires repeated sequential runs or an explicit catch-up controller; schedule frequency alone does not clear many pending days quickly.

The monthly-close experiment should begin only when a detailed billing file is present. It must record the provisional BEFORE state, validated SOURCE and stored AFTER state for the same month and environment. The technical reconciliation passes only when AFTER row count matches SOURCE, their BilledCost difference is within tolerance and critical-null count is zero. This demonstrates faithful replacement of the billing candidate; it does not determine whether a nonzero SOURCE-minus-BEFORE difference is financially justified. DEV should pass before PROD is promoted, and a failed or interrupted promotion should be replayable without changing a previously closed month incorrectly. The dedicated Classic orchestration and its test results are still pending.

### 7.4.4 Limits of current validation

The currently indexed local tests demonstrate generator consistency and cloud-code behavior outside Databricks. The Belgium plan provides a credible progress record for historical cloud loading, but without primary artifacts the thesis cannot report exact cloud row counts, durations, DBUs or application behavior as measured results. No controlled Serverless/Classic performance or cost comparison follows from the presence of both configurations. RLS and direct SQL-access rules require separate access tests. These limits are carried into chapters 8 and 9 rather than filled with estimates presented as observations.

## References

Microsoft. (n.d.). *Use Performance Analyzer to examine report performance*.

<https://learn.microsoft.com/en-us/power-bi/create-reports/performance-analyzer> (accessed 29 September 2026).